

Stateless vs. Stateful Aggregation in Orbital Federated Learning: Project Status Report

Xiao Luo

Department of Computer Science, Purdue University
CS 59200-EAI research project
Instructor: Prof. Bharat Bhargava

October 7, 2026

As of October 7, both preparatory tracks are done: Track A, an end-to-end tracer bullet, and Track B, the orbital oracle and energy ledger. I am now writing the aggregation substrate and its two extreme bindings, which every later experiment depends on. Section 2 re-plans the nine weeks left before the final presentation. It narrows the proposal’s scope, mostly because of what the tracer bullet showed.

1 Progress to date

1.1 Design and formalization

Before writing any system code, I settled the design and drafted the paper’s introduction, background, and related-work sections. The architecture splits in two. Its control plane holds a visibility and resource oracle and the placement controller; its data plane holds the aggregation state layer and the serverless runtime. All four placement patterns sit on one shared state substrate, which makes placement the only factor the study varies. No other design decision shapes the experiments as much. On the formal side, I wrote out the energy-ledger equations, a proof sketch that the aggregation step forms a commutative monoid, and the placement problem as a constrained optimization with hysteresis. The evaluation runs at three fidelity tiers. L1 is full-stack emulation at 10 to 40 nodes, L2 is lightweight emulation at 100 to 300, and L3 is trace-driven simulation at Starlink Shell-1 scale. The instrumentation schema maps every symbol in the optimization to a telemetry event. I have frozen the interfaces and data schemas in code.

1.2 Track A: the tracer bullet

A tracer bullet is the thinnest slice that passes once through every layer. My tracer ran four satellites and one ground station at 550 km as Celestial [1] microVMs, each with a real function runtime inside. I invoked functions only inside contact windows computed in advance, pushed one FedAvg round on MNIST through the stateless binding to a ground store, and logged every event under the instrumentation schema. The systems slice passed in three runs with identical configurations. The aggregate moved end to end in its wire format, every commit was durable, no client ran outside its contact window, and the telemetry had zero schema violations. Rounds closed between 4,761 and 4,861 s, which is 2 to 4% above the floor that contact geometry sets for this scenario (about 4,670 s). Two measured costs account for the gap. After each acquisition of signal, 78 to 163 s passed before the first invocation succeeded, or 15 to 32% of a 511 s contact window, and activating the aggregator took 18 to 36 s. Knative does run inside Celestial’s microVMs. The open question

was what the satellites should run given that readiness delay, and a paired benchmark settled it: a thin FaaS shim on the satellites, with Knative kept on the ground segment as the fidelity anchor.

The tracer also caught two errors before I built anything on top of them. The first was that the model did not learn. Every run reported 7.86% accuracy on ten-class MNIST, which is below chance, and the number was bit-identical across six runs on two runtimes. Since it did not change with the runtime, it could only have come from the seed. After the fix, the same round reaches expectation. Repeatability checks had let the bug through, so I added a learning floor to the tracer’s acceptance gate: on IID MNIST, the committed model must reach 0.80 accuracy and beat the initial model by at least 0.30. The second error was in my runbook. It set a 72-minute round target that no implementation could meet. Because a ground station turns with the Earth, each satellite’s passes recur at the synodic period rather than the sidereal one, so the four satellites’ passes arrive 25.6 minutes apart and the last begins 76.8 minutes after the first. I now derive the round deadline from the contact schedule.

1.3 Track B: orbital oracle and energy ledger

Track B produced the oracle and the ledger as a standalone, pure-Python library that does not depend on the emulator. The oracle propagates TLEs with SGP4 (via Skyfield), extracts contact windows with padding for TLE drift, and computes the features the placement controller will read, such as contact density and ISL chain persistence. Two separate processes given the same TLE file and configuration produce byte-identical schedules. The ledger implements the battery recurrence, the 50% depth-of-discharge (DoD) cap with forced payload shutdown, the charging-current ceiling, and thermal throttling, all from the model and in-orbit measurements of Li et al. [2]. Its golden tests derive their expected values in closed form instead of recording them from earlier runs. If the code drifts from the equations, the tests catch it. One test reproduces, in a synthetic configuration, the regime Li et al. found when they replayed their orbital telemetry with ten times the harvest. In that test, 80% of the harvest goes to waste and the battery still crosses the DoD cap at hour 15, because it cannot charge faster than its current ceiling allows. The two tracks also agree on geometry. At 550 km, the pass duration Celestial produced matched the oracle’s closed-form value of 511 s to within 0.02%.

1.4 In progress: AggState and the two extreme bindings

AggState is the substrate all four patterns share. It is a round-versioned store of partial aggregates, each tagged with its round, contributor set, and staleness. Merge is a weighted-sum fold, and commit is a durable compare-and-swap. The stateless binding keeps nothing on the satellite; every aggregator invocation fetches the model from a ground store and commits back to it. The stateful binding pins the state on a sink satellite, records each round delta in a write-ahead log, and mirrors each delta to a shadow replica on the next satellite in the plane, with a checkpoint to the ground on every pass. Recovery promotes the shadow and replays the log. I am building these two first because they bound the design space and exercise all of the durability machinery. Hierarchical and gossip aggregation compose from the same partial-merge primitive. M1 below says when the two count as done.

2 Plan to the final presentation

The tracer bullet changed how I am planning the rest of the semester. It showed that a pipeline can pass every systems check and still not learn, so no binding reports a number until it clears the M1 checks. It also put a price on experiments. Celestial emulates in real time, and the four-satellite tracer round took about 80 minutes of wall clock, so the scarce resource is emulation hours, not engineering time.

Change of scope. The proposal planned all four patterns at small scale for the intermediate report and, for the final presentation, the full matrix at medium and Shell-1 scale with a validated selector. I am narrowing the semester to the two extremes. Those two get three seeds per cell, fault injection, confidence intervals, and a rule that no difference smaller than seed variance counts as a result. Two verified bindings with replicated runs say more about placement than four bindings run once each. The cut follows the order the project fixed in September (gossip first, then hierarchical; the two extremes and the selector are never cut). The selector stays in the chapter. It moves to spring because a four-way selector needs all four patterns to fit and held-out scenarios to test, and neither exists yet.

Milestones. The dates assume the presentation falls in the last week of classes, December 7 to 11, and leave room for fall break (October 12 to 13) and Thanksgiving (November 25 to 28). If a milestone runs late, I will push the later dates rather than relax its checks.

- M1 *Substrate and both bindings, verified (October 7 to 30).* No experiment starts until five checks pass. (i) Property-based tests confirm that random groupings and orderings of the same updates commit the same model, up to float64 round-off. (ii) Both bindings must commit the same model from identical inputs. That equivalence is what makes the comparison a one-factor experiment. (iii) Everything the pipeline reports gets a sanity floor: learning (the floor above), energy (per-round ledger energy must match a hand calculation from measured execution time and bytes moved), and the network (shaping counters must show that traffic crossed the emulated links). (iv) I inject crashes into the stateful path at every step of merge, log append, and commit, then check that recovery neither loses nor double-counts an update. (v) A per-binding overhead microbenchmark has to show the substrate costing about the same under both bindings. If it does not, any difference I report could be an artifact of my implementation.
- M2 *Head-to-head at L1 (November 2 to 21).* I emulate 10 to 40 nodes in a Shell-1-like geometry (550 km, 53°). The Track B oracle schedules every invocation. MNIST covers the full grid of binding $\times$ churn (low, high) $\times$ data distribution (IID, non-IID), with three seeds per cell, plus injected satellite loss and link drops for both bindings. CIFAR-10 with ResNet-18 runs a fixed number of rounds at the two cells where MNIST separates the bindings most and least. I do not train it to a target accuracy. It is there to measure the cost of moving an 11M-parameter model. The metrics are the proposal’s: time to target accuracy, wall clock per round, aggregation p50/p95 latency, staleness, bytes per useful update, recovery time, state read/write amplification, and energy per round. If hours run short, I drop CIFAR-10 cells before any cell loses a seed.
- M3 *Analysis and write-up (November 16 to December 4).* An L1 run lasts hours, but battery depletion under sustained onboard compute plays out over tens of hours [2]. To bridge the two, I will feed the per-round costs measured in M2 into the ledger, run it over multi-day contact schedules for the same configurations, and label the output as a projection. Then I will build the first slice of the regime map. For each cell of the M2 grid, it shows which binding wins on each metric and by how much, with confidence intervals from the seeds, and whether the

multi-day energy projection moves the boundary.

What the final presentation will show. The verified substrate and both bindings, with the M1 evidence; the L1 comparison under fault injection; and the first slice of the regime map with its energy projection. Together these answer the hypothesis for the two extremes: either a regime boundary separates stateless and stateful at L1, or none does and the data show which features failed to separate them. If M2 finishes early with every check passing, the hierarchical binding joins the L1 runs.

Deferred. Hierarchical and gossip bindings; the L2 and L3 tiers and their cross-validation against L1; EuroSAT; the four-way selector with hysteresis and its held-out validation; and the hardware-in-the-loop energy measurement on a Jetson or Raspberry Pi.

Risks. The hardest part of M1 is the stateful binding’s failure semantics, especially a sink that fails mid-commit, so M1 stays open until the crash-injection tests pass. The other risk is emulation hours, and the priority rule in M2 covers it.

References

- [1] T. Pfandzelter, D. Bermbach. Celestial: Virtual software system testbeds for the LEO edge. ACM/IFIP Middleware, 2022.
- [2] Q. Li et al. Exploring real-time satellite computing: From energy and thermal perspectives. IEEE RTSS, 2024.